The Law of Graphic Equalisation
Statement

Matthew Parslow
Independent Researcher

Draft — draft-2026-08-30.1
30 August 2026 • statement

STATEMENT. This is the theory; the companion *Law of Graphic Equalisation* (Theory) is the case for it. What is missing here is not detail but *argument* — the removals that establish how little is required are the long paper's bulk and its actual work, so anything below that a reader wants shown rather than asserted is shown there. Calling this a short form, as an earlier version did, described it by its length instead of by what it does. Every definition and proposition below is pulled from the long paper at build time, so the two cannot drift.

Abstract

One minimal structure is proposed as underlying phenomena usually studied apart. This statement gives the structure, the frame machinery through which anything reaches it, what happens when a write is refused, the single prohibition it carries, and the commitments on which it fails. It argues for none of them. The removals that establish how little is required are the long paper's bulk, and a reader who doubts a particular claim here should expect to be persuaded there and not by this.

1 The substrate

The substrate is structure, the transformations operative upon it, and a grammar of which relations may exist. Only the first is required for there to be anything; the other two are what a world in which anything *happens* costs. Nothing beyond the three is primitive, and anything else exists where a graph happens to represent it.

Definition 1.1 (Graph)**.** *A graph is*

$$\boxed{G = (g, r, \Gamma),}$$

where g is represented state, r is operative transformation structure, and Γ is the admissibility grammar governing graph-originated transformations.

Definition 1.2 (Grounding)**.** *A* grounding *is a particular assignment of rule content and initial conditions to the substrate — one (g, r, Γ) with its laws and its starting configuration fixed — together with the domain it is taken to model. The substrate is what every grounding shares; a grounding is what any one of them adds. Claims made about the substrate are accordingly claims about all groundings, and are correspondingly weak; claims made within a grounding carry that grounding's contingent rule content and are where this account's falsifiable commitments mostly sit.*

Definition 1.3 (Admissible continuation)**.** *A continuation δ belongs to $\mathcal{P}(G_t)$ iff it is expressible in the current graph vocabulary, satisfies the relevant antecedent conditions, and is admissible under the operative grammar for the transformation class in question.*

Definition 1.4 (Collapse)**.** *Collapse is the frame-relative passage from a pre-realisation possibility structure to realised continuation:*

$$\boxed{\mathcal{C}_t : \mathcal{P}(G_t) \rightsquigarrow G_{t+1}.}$$

It is not identified with deliberative choice.

The Law is that a graph's admissible continuations are fixed by its present structure, and that realisation selects one of them, which then becomes the antecedent for what follows.

Definition 1.5 (Residual)**.** *The residual is*

$$\boxed{\epsilon = \delta^R \ominus \delta^P,}$$

where $\ominus$ *is the grounding's difference relation preserving relevant discrepancy.*

Remark 1.6 (Only g is required; the triple is what a *running* world costs). The triple is not the minimum and this paper has previously implied that it was. Strip r and Γ and something remains: structure, with relations among its parts. Nothing *happens* in it — no transformation, so no collapse, no residual, and nothing it individuates — but it has not thereby become nothing, and it has not stopped being individuable by something else. What r and Γ buy is *occurrence*, and everything this account is about — change, collapse, observation, cost, experience — requires them. The triple is the price of a world in which anything happens rather than the price of there being anything, and Remark 6.2's "only that rules run is compulsory" should be read in that narrower sense.

A distinction is doing the work there. Being a frame is being individuated (Definition 2.1), and individuation is performed by the container, not by the thing: a boundary belongs to the frame that draws it. So a structure with no rules of its own is still a frame — something else instantiated its boundary — and what it lacks is not framehood but the capacity to individuate anything itself. The earlier claim that stripping r leaves "no frame" was wrong, and it would have invalidated the companion result that reasons from a rock's being a frame to the universe's being one.

2 Frames

A frame is how anything is reached at all: there is no view from nowhere, so every claim is a claim made in some frame, and the frame is not a container holding contents but the relation that demarcates them.

Definition 2.1 (Frame)**.** *A* frame *is an instantiated boundary relation — the edge* $\mathcal{B}_{W:G}$ *itself, and not the subgraph* G *it demarcates. The subgraph is the relatum; the frame is the relation. To individuate is to instantiate such an edge, which is why individuation is per frame rather than a fact about the graph.*

Corollary 2.2 (Frame and dimension are one edge type in two roles)**.** *A dimension is the same construction read from the other end. The boundary* $\mathcal{B}_{W:G}$ *is a dimension of* W *— one of the properties along which* W *separates what it contains — and each dimension* W *instantiates is a frame to the values it separates. So* frame *and* dimension *name two roles one edge type plays relative to another edge, not two kinds of thing, and the asymmetry between them is positional in the same way that the asymmetry between a parent and a child is.*

Proposition 2.3 (One interface, two gates)**.** *A situated graph meets its containing frame through a single relation, and what an observation from that frame delivers is not distinct in kind from what the containing frame imposes: to a subgraph, being observed* is *being deformed by its*

container. That one relation is gated twice, by the same kind of admissibility grammar on its two faces — the aperture grammar Γ_a *on the input and the* change grammar Γ_c *on the output:*

$$\boxed{\underbrace{\textit{aperture}}_{\Gamma_a} \longrightarrow \underbrace{\textit{rules}}_{r} \longrightarrow \underbrace{\textit{change}}_{\Gamma_c}.}$$

Γ_a *limits, by grain, which of the container's deformation enters the rules;* Γ_c *determines which of the rules' realised deltas cohere into the container and which remain as residual (Proposition (long form)). Both are faces of the frame's one grammar* Γ*, and instantiate for a specific container* W *as* Γ_{W_a} *and* Γ_{W_c}*; it is* Γ_{W_c}*, the change face, that the residual proposition uses. Execution runs between the two.*

Definition 2.4 (Aperture)**.** *A frame's* aperture *is its input gate* Γ_a *together with what that gate fixes: which dimensions it can separate along at all, how finely it separates within each (its resolution, per dimension, for which this paper also uses* grain*), and how far along each it reaches. The three are properties of one edge — existence, grain, and extent — and are named separately because they vary independently, not because they are different kinds of limit. What falls outside an aperture on any of the three is* absent *to that frame rather than held-but-unresolved.*

Definition 2.5 (Change grammar)**.** *A frame's* change grammar Γ_c *is its output gate: it determines which of the realised deltas its rules produce cohere into the containing frame and which do not. What it declines has entered the frame and been operated on, so it is* residual *(Definition 1.5) rather than absent, and it persists in place rather than being discarded.* Γ_a *and* Γ_c *are the two faces of one grammar* Γ *on one boundary edge, and the asymmetry between them is which direction they gate, not what kind of thing they are.*

Definition 2.6 (States of a dimension)**.** *For a frame* $\mathcal{F}$*, a dimension is in exactly one of four states:*

$$\boxed{\text{NULL} \neq \text{undetermined} \neq \text{undisclosed} \neq \text{resolved}.}$$

NULL*: the dimension is not instantiated; there is nothing to have a value.* Undetermined*: the dimension is instantiated and* no value has been realised *— collapse has not occurred.* Undisclosed*: a value has been realised but is not held by this frame.* Resolved*: a realised value, held.*

3 Economics

Cost attaches to writes rather than to holding, and what a write costs is set by whatever runs the rules rather than by any universal.

Proposition 3.1 (Cost characteristically reduces to the write)**.** *In the substrate as characterised here, the boundary relation itself is free; the standing structure persisting is free; the operation that costs is a delta that writes over that structure, and* r *is what writes. Free here means information-preserving rather than absent — evaluating a rule is itself a write, and is unpaid because it destroys nothing (Remark 3.3). To that extent*

$$\boxed{\textit{cost is paid by rules.}}$$

This is a characterisation of the substrate as specified, not a necessity of graphhood, and the writeability status it describes is not required to be uniform (Remark (long form)).

Definition 3.2 (Well-formedness)**.** *A graph is* well-formed *to the degree that it pays for nothing that does no work for it. Three costs bear on that and they behave differently, so the notion is graded along each rather than binary.*

Writes *are paid when made (Proposition 3.1), which prices doing.* Retention *is nearly free per unit, which is why an unused distinction is not immediately pruned.* Search *is paid on every operation over what is held and scales with how much is held, so inert structure is cheap to keep and expensive to keep in quantity.*

The fourth is the worst and is not a variant of the third. A write may land on structure that cannot affect anything — a modification to a branch nothing reaches. It is paid in full: decided, made, checked, risked. It returns nothing. And unlike a search, which is expensive and then yields, this failure is silent*: the graph has spent, has no effect to show for it, and no signal that the effect is missing. What it acquires instead is a false model of itself, in which something was changed that was not — residual it cannot see, produced by its own inert contents, and the share of writes exposed to it grows with how much is held.*

So a graph does not become ill-formed by carrying something useless. It becomes ill-formed by carrying enough that finding the useful parts costs, and enough that its changes start landing where nothing happens.

The third cost is where aperture enters, and it is why the same contents are well-formed for one graph and not another. What search costs depends on how finely the graph can discriminate within what it holds: with structure that separates the relevant from the rest, the cost grows slowly with size; without it, every operation is over everything. That discriminating structure is itself held and itself earns its keep, so a well-formed graph is one whose aperture over its own contents keeps pace with their growth.

Removal is therefore not free either, being a write. Pruning happens when accumulated cost exceeds the one-off cost of removal, and the fourth cost is what usually triggers it: search cost is diffuse and tolerated for a long time, while a change that demonstrably did nothing is a discrete and memorable event. Systems clean up after being burned rather than after slowly noticing. Well-formedness is graded rather than binary; it is relative to what work *the graph is doing, so a graph well-formed for one purpose is not thereby well-formed for another; and it is not enforced by the substrate. An ill-formed graph runs. It pays more to.*

Remark 3.3 (Evaluation is a write, and that is why it is free)*.* The economics above can be stated in a way that does not survive contact with the machinery, and the objection is worth answering rather than rewording. If running a rule that does not fire is a read, and reads are free, where does the check go, and how does the substrate know to run the next one?

Computing answers it without any appeal to physics. `read()` is an operation, and it returns data; the returned datum has to be somewhere to be used, and in a substrate that is graph all the way down there is no return channel outside the structure, so the value materialises as structure. A read therefore writes by being a write, and the write is the return. An operation that returns nothing is not a read of anything, which is why the free case exists and is exactly the empty one: a read whose return is never used has read nothing, and a compiler will delete it. Real machines make the same point expensively — a read leaves traces in caches, predictors and translation state, and those traces are measurable, which is why timing attacks recover secrets from a victim whose only secret-dependent operation is a read (long form) and why an entire class of speculative attacks was available to designs built on the assumption that rolled-back speculation leaves no trace (long form), where the read is the vehicle rather than the premise.

Evaluation is a write in that plain sense: a match outcome, a position in an ordering, and the fact of having tried are returned values, and returned values are structure. What does not follow is the price. A rule's cost is set by whatever supplies its primitives and is inherited along provision (Section (long form)), and Landauer's bound (Remark (long form)) is a floor in one substrate rather than the price in any. This is also why continuation presents as free from inside: a well-formed runner does not expose itself (Proposition 6.4), so its charge is below the frame's aperture and is not an in-frame event at all. In-frame nothing happened; in the running substrate something was paid; both hold, of different frames.

Two things follow for the rest of these papers. Reversibility is a structural fact about a

map and cost is a fact about what supplies a primitive, and conflating them is what the earlier phrasing did — separating them is what lets the physics companion recover unitarity, which needs only the first, since a no-write was never the right description of an evolution that plainly changes the state and is reversible because it is injective. And the residual named in Remark (long form) attaches here: information-preserving evaluation accumulates the history it preserves, so a frame running a long time without collapsing has deferred a cost into retention rather than escaping one. What is free is not the running, only the not-yet-forgetting — and free only relative to a frame that cannot see the bill.

Proposition 3.4 (Primitives leak, and complete provenance is still unrecoverable)**.** *Leaking is ordinary rather than exceptional. A primitive's timing discloses the structure beneath it, which is why measuring how long a read takes recovers what a cache holds (long form); an arithmetic that overflows at a particular magnitude discloses a word size; a rounding discloses a representation; and a performance cliff at a particular size discloses a page. Most abstractions leak, and this one is not special.*

How a leak reaches a frame at all is worth stating, since what falls outside an aperture is absent *(Definition 2.4) and an absent thing cannot disclose anything. It does not arrive by perception. What arrives is an ordinary in-frame observable on a dimension the frame already instantiates — a duration in its own units, a value at a boundary — and what the frame does with it is* infer*. The cache is not perceived; the timings are, and the cache is concluded from them.*

What does not follow is that provenance can be recovered entire. Every act of recovery is itself performed by primitives, so a graph inferring what lies beneath one operator does so using others whose own provenance is not thereby in view, and there is no position from which the whole supply is inspected at once. The result is the standing shape: partial always, complete never *— leaks give a graph real and sometimes decisive knowledge of what supplies it, and never the closure of that question.*

4 When a write is refused

An inadmissible write yields no continuation in that frame. What happens after that — whether anything propagates, whether a value is produced instead — is rule content and not substrate, so the exception reading below is an analogy and the systems it names are policies rather than discoveries.

Remark 4.1 (Decoherence is one operation; the frames are what differ)*.* Two strings make the structure visible. $2 + 2 = 5$ is a perfectly coherent set of symbols: it parses, every token is admissible, the arrangement is grammatical. *Two particles smash into each other at* .99*c* is coherent in the same way. Both pass the aperture and fail at the change grammar, which is the two-gate structure of Proposition 2.3 in its sharpest case — what is admitted as *represented* need not be admissible as a *change*.

What follows is not a partial merge. The collapse realises nothing, and the result is NULL — which is not the same as *absent*: absent is never represented, whereas this is represented, admitted, and yields no continuation.

The obvious objection is that $2 + 2 = 5$ has a truth value and that value is false. It does not, and the notation already says so. $2 + 2 = 5$ *is not false;* $2 + 2{==}5$ *is false* — one sign writes and the other reads, and computing has kept them apart in its surface syntax for exactly as long as it has had both. The comparison is an ordinary in-frame operation returning an ordinary in-frame value. The assertion is a write, and in a typed setting it does not evaluate to anything at all: it is refused before execution, because nothing admits an assignment to a value. Rejected by the grammar rather than run, with no continuation taken from it.

So the NULL belongs to the write, and the familiar falsity belongs to a different operation that is spelled differently. That the two are easy to confuse is not a quibble either — writing the write where the read belongs is among the most persistent defect classes there is, guarded against by compilers and linters because the cost of the confusion is high. The distinction is load-bearing wherever both operations exist, which is the point of Section (long form) arriving from the other direction. And the residual is not a local remainder. Admitting $2 + 2 = 5$ into arithmetic does not cost a repair somewhere; it costs the grammar's restrictiveness, since a grammar that admits a contradiction admits everything and thereby stops restricting anything. So the residual is *everything in the superframe*.

The two companion papers instantiate this at their own grains, and the word is used in both. The physics companion separates the sense used here from the standard environmental one and states what relates them; the companion on mind reads death as this operation with the frame set to a person. Neither is required for what follows.

The point is what does *not* vary. Decoherence is one operation. It does not come in kinds and it does not come in degrees, and the two cases above fail identically. What differs between them is the frame — arithmetic in one, kinematics in the other — and *the frames are what make the cases different, not the failure*. "Everything in the superframe" is a constant expression whose value is frame-relative, which is Axiom (long form) applied to incoherence rather than to observation.

This is worth stating because the obvious gloss is wrong in a specific way. One is tempted to rank them — arithmetic lies deeper than kinematics, so denying it costs more — and depth is real, but it is a fact about a web of commitments and therefore about a frame, never about the decoherence. The temptation is to read a magnitude off a difference that belongs elsewhere. That no distinction of *kind* survives here between the analytic and the empirical case is Quine's conclusion (long form); the present account reaches it without needing his gradient, since the failure is uniform and only the frame is indexed.

The mechanism for that indexing settles what would otherwise look like a family of competing answers, and the useful way to see it is by analogy with exceptions — an analogy, because the substrate fixes far less than the analogy suggests. What follows from admissibility alone is only this: an inadmissible write yields no continuation *in that frame*. Everything downstream of that — whether anything propagates, whether the container is notified at all, whether a value is produced instead and which one — is rule content, and rules here are contingent throughout. There is no substrate-level default.

Read as the analogy it is: a frame that rejects a write raises, the failure goes one frame up and up again until something handles it or nothing does, and unhandled to the top the whole thing stops, which is what "residual of everything in the superframe" describes. A frame may instead cohere the failure to a value — false, a gap, NaN, null, or a contradiction held without explosion — and any of these is available, as installed machinery someone had to put there. Nothing about the failure selects the value; the frame's rules do.

The reason to insist the framing is arbitrary is that computing itself does not agree on it. Exceptions propagate by default and are handled on the marked path; return codes, result types and option types make the check explicit at every call and propagate nothing on their own; hardware may trap or may set a flag and continue. These are not approximations of one true discipline. They are different rule sets for the same underlying fact, which is only that something did not work.

Which reorganises the prior art rather than accumulating it. *These systems are handlers.* Bochvar's infectious junk value, the neutral free logics' gaps for empty terms, Reichenbach's third value for the quantum indeterminate (long form), paraconsistent absorption (long form), relevance logic's refusal of ex falso on grounds of relevance (long form) — each is a policy for what a frame coheres a failure to, and that they are mutually incompatible while each being defensible is evidence they are choices rather than discoveries.

The demonstration is in silicon and is not a metaphor. IEEE 754 gives floating point a *value* where integer arithmetic *traps*: 1/0 is an infinity in one and a fault in the other, 0/0 is NaN, and NaN then propagates through every operation containing it — weak Kleene, in hardware, by deliberate standardisation (long form). One machine, two policies, chosen per type. The failure did not choose; the committee did.

The economics is therefore weaker than an earlier version of this remark claimed, and the claim it made is a good example of the error it was warning about. It said propagation costs nothing while a handler is purchased, so that a total residual is the unmarked case. That holds *under the exception convention* and not otherwise: where failure is carried in return values the check is explicit and paid at every call, and it is the propagation rather than the handling that costs. Which side the cost falls on is the convention's, not the substrate's.

What survives generally is only that a policy of any kind is held structure and is paid for under Proposition 3.1, so a frame that bounds its residuals holds more than one that does not, whatever discipline it uses to do it.

The physical case at the largest scale makes the purchase visible, and it is the same shape rather than an illustration of it. A heavy nucleus sits near the limit of what holds it together; add a neutron and the configuration cannot be held, so it does not continue as that nucleus and the binding energy goes to the container. A reactor and a weapon are then the identical failure with the handler installed and with it removed — control rods, moderator and containment are held structure, paid for continuously, and the whole of their function is to bound a residual that is otherwise total. What the Manhattan Project's design problem amounted to was keeping an unhandled configuration assembled long enough for the propagation to finish, which is the cost of *defeating* a handler rather than of installing one.

Two things there are worth marking rather than absorbing. The handler is viable only because a small fraction of the neutrons are delayed — well under one per cent — which supplies a timescale it can act within; a frame whose failure propagates faster than its handler can run has, effectively, no handler, which is a constraint on the purchase that the logical cases do not exhibit. And criticality is the condition where each raise produces more than one further raise, so the residual does not merely ascend a frame but multiplies. That is a genuine difference from an exception unwinding a stack, and the substrate accommodates it only because residual is per-rule (Proposition (long form) and the paragraph following it) so one failing write may leave several. It should be recorded as a difference and not smoothed into the analogy. It is also where this remark meets the Narrativium paper from the other side, since a process whose products exceed their cause in number is that paper's spend-and-gain iterated above unit gain — the same ratio, arrived at through failure here and through capacity there.

Almost all of this is prior art, in a literature this account did not come from, and the correspondence is close enough to be worth stating precisely. That a string may be syntactically well-formed and yet fail is Ryle's category mistake, and he used such failures for the purpose this remark puts them to — delineating the categories, the failure showing where a boundary runs (long form). That the result is neither true nor false but a third thing is the truth-valuelessness position. And the propagation — residual of everything in the superframe — is exactly the long form's three-valued calculus, in which a compound takes the junk value if any constituent does, so nonsense is infectious rather than local. the long form named the resulting system the logic of nonsense and made it *paraconsistent*, which combines in one system what the paragraphs above treated as two frames; the separation of a theory being inconsistent from its being trivial is likewise established, and is what the economic reading here restates (long form).

One further correspondence is not a debt but evidence for a different claim in this paper. Computing reached the same machinery independently and for its own reasons: ill-typed terms are refused before execution rather than evaluated to anything (long form), and denotational semantics gives divergence a value $\bot$ whose strict functions satisfy $f(\bot) = \bot$, so bottom propagates through any compound containing it (long form). That is Bochvar's infectious value,

re-derived some three decades later, by people solving computation rather than the semantic paradoxes. Two fields under different binding constraints arriving at one arrangement is exactly what Proposition 7.2 predicts, and it is offered here as an instance of that rather than as support for this remark, which needs none.

What is not taken from that literature is the frame-indexing. The formal systems fix an infectious value and derive its behaviour; the claim here is that the same failure yields the appearance of falsity in one frame and of nonsense in another, so the taxonomy separating $2 + 2 = 5$ from a category mistake is itself the frames differing rather than the failures. That is a claim about where the distinction lives, not a new logic, and the logics above are the machinery it would be written in.

The second string carries a further lesson and is a better instance than it first appears, because it is not in fact false. A closing speed of $1.98c$ is unproblematic, being no object's velocity, and the relative velocity is $(0.99 + 0.99)/(1 + 0.99^2) \approx 0.99995c$, under c. The incoherence arises only when the sentence is evaluated *in one frame by addition* — an in-frame operation applied to a quantity that lives between frames. It fails as a category error rather than as forbidden physics, which is the in-frame/cross-frame discipline of Section (long form) appearing as a coherence verdict.

Remark 4.2 (A handler can be abused, and the abuse is invisible from inside)*.* If a handler decides what a frame coheres a failure to, it can be set to cohere it to anything, including success. Computing knows this as the swallowed exception — the catch that discards what it caught, the broad handler that reports normal completion — and it is not a defect in the mechanism but the mechanism used against its purpose.

What that does is precise in this account's terms, and it is already costed. Swallowing does not undo the inadmissibility: the containing grammar still refused, and the write still did not happen. What is removed is the frame's *record* of that, which leaves the frame holding a model of itself in which something occurred that did not. That is exactly the fourth cost of Definition 3.2 — expenditure returning nothing, with the failure silent — and the swallowing handler is the mechanism by which a false self-model is manufactured rather than merely risked.

The stronger form follows from the same premise and is worth stating plainly. *If the handler determines the outcome, then causing a failure is a way of choosing the outcome.* One need not make the preferred continuation admissible; one can make the alternative fail to cohere and let the policy deliver the result. Failure becomes an instrument rather than an accident, and the move is recognisable wherever a procedure has a default: the party who prefers where the exception path lands has no reason to argue on the merits.

Detection is available but never free, and the exact shape of that is worth stating, because the obvious formulation overshoots. It is not that a frame cannot detect a swallow: a frame holding a record of the attempt separately from the handler's report can compare the two and find the divergence, and well-built systems do exactly that. What is unavailable is detecting it *from the handler's own report*, which is self-consistent by construction. So the audit is installed structure, paid for like any other, and a frame that has not bought one has no access to the discrepancy. The audit is itself a policy, abusable in the same way one level in, so what the purchase buys is relocation rather than closure — the same regress the provenance commitment names, terminating only in a containing frame. That is Axiom (long form) arriving where it is least convenient, since the frame with most reason to detect this is the one whose own machinery is doing it. The silence of the fourth cost has this structure, and so does the presented-versus-produced divergence the companion paper on mind treats at the scale of one frame acting on another.

Remark 4.3 (What never cohered is absent; what cohered and stopped leaves a hole)*.* A distinction the preceding material needs and has not drawn. Something that never cohered is simply *absent*, and absence is free — nothing points at it, nothing expected it, and no frame is worse off for it. Something that cohered and then stopped is a different object: it leaves *a space where something*

was, and the difference is that other structure came to depend on it while it held.

The consequences of the second are underdefined, and underdefined in a specific way rather than merely unpredictable. What breaks is not the removed structure — it is gone and has no further behaviour — but every relation elsewhere that resolved through it and now resolves through nothing. Computing has the sharp case and calls it exactly that: dereferencing what was freed is undefined behaviour, and its mildest form is a wrong answer while its worst is unbounded. A variable that was never assigned and one that was assigned and released fail differently, and only the second can corrupt something that was correct.

The reason this cannot be surveyed in advance is structural rather than practical. *A thing does not carry a record of what depends on it.* The dependencies live in the frames holding the references, not in the referent, so from the position of the thing being removed they are not visible at all — Axiom (long form) again, in its most operational form. Knowing what would break requires the containing frames, and asking them is the expense.

Which yields three states rather than two, and the third is the one that costs. Absent: nothing held, nothing owed, free. Vacated with the references left standing: a hole, unpaid, with consequences determined by whatever the dangling relations do next. And vacated with the absence *represented* — a tombstone, a cascade rule, a retained count of dependents — which resolves cleanly and is installed machinery paid for under Proposition 3.1, the deletion-side member of the handler family of Remark 4.2. Reference counting is not a fact about references; it is a purchase.

This is also the sharper form of the removal cost argued later for genomes and dead code (Remark (long form)). The reason deletion is expensive is not mainly that the excised structure might turn out to do something. It is that the survey licensing the deletion is a question about *other* frames, obtainable only from them, and a confident remover is someone who has not paid for it. Chesterton's rule about the fence is this stated as advice (long form); the ecological version, where removing one species collapses relations that were not obviously routed through it, is the same structure with the dependents alive (long form).

Remark 4.4 (Termination propagates along supply, not containment)*.* A consequence of primitives not exposing their provenance (Proposition 6.4) that the preceding material assumed away. A subgraph's operations need not be supplied by the graph that contains it: a primitive may come from arbitrarily far up and be passed through, and nothing in the subgraph can tell. So if a graph G sits inside W while running on primitives supplied from beyond W, then W ceasing does not stop G. *There is no reason it cannot continue*, because what ran it was never the thing that stopped.

Two relations that ordinary usage runs together are what this separates: being contained by something, and being run by it. They usually coincide, which is why the distinction is easy to miss, and the coupling result already says they need not (Proposition (long form)) — a graph fused to what supplies it has no independence, a loosely coupled one has all of it. Survival of a container's termination is therefore a question about *coupling* and not about containment, and it is arrangeable: a frame that wants to outlive its container can be built so that its supply comes from higher up, which is a purchase like any other.

The rule this yields is short. *Termination propagates along the supply relation, not the containment relation.* Downward it need not reach at all — a container may stop while what it held runs on. Upward the supplier's stop is total: everything running on primitives from a graph stops when that graph does, at whatever containment distance and with no handler available, since the operations a handler would run are the ones withdrawn.

And it refines the previous remark rather than sitting beside it, because it splits the dependents into three kinds, not two. A dependent that merely *refers* to a vacated structure is left dangling and keeps running, wrongly. A dependent *wholly run by* it simply stops. And — since supply is a set rather than an edge, primitives being drawn from wherever they are drawn from — a dependent may be *partly* supplied by what stopped, in which case it keeps running

with an incomplete set of operations and its behaviour is undefined.

The third is the worst of them and is worth separating out, because it is the silent one. Such a frame does not stop and does not visibly break; it runs until it reaches a rule requiring an operation that is no longer there, and *it cannot know in advance which rules those are.* There is no dangling edge to find, because a withdrawn primitive leaves no structure behind pointing at nothing — the operation is simply absent. And the frame cannot audit for it, since provenance is not exposed (Proposition 6.4): there is no procedure by which it asks which of its own operations came from the thing that stopped. So the loss is discovered only by attempting it, which is Remark 4.5 arriving with nothing to be learned that was worth the price.

That places it in the family this paper keeps returning to. A partially supplied frame holds a model of itself in which it can do things it can no longer do, which is the false self-model of the fourth cost (Definition 3.2) and of the swallowed handler (Remark 4.2), arrived at a third way. Graceful degradation is the name for buying one's way out of this, and it is worth noting that it is a purchase: the *unbought* behaviour of a partially supplied frame is not graceful, it is undefined.

Remark 4.5 (Forcing a failure is the instrument, and handlers blunt it). Remark 4.2 treats a caused failure as a way of choosing an outcome. The same act has a second use which is not abuse at all, and it is how most of what is known about any grammar was found: *a frame that never fails discloses nothing about what it admits.* Admissibility is invisible from inside its own range, since everything works; the boundary is legible only where it is crossed. So the way to learn a rule is to force the failure and read the residual.

This is not a marginal technique but the standard one across unrelated fields, which is what a generative claim should predict. Genetics removes a gene and observes what stops (long form); neuroscience reads function from lesions; machine learning ablates a component to price it; software injects faults to check that anything notices (long form). And physics builds the largest instrument there is on exactly this principle — an accelerator spends energy to push interactions onto one step, where the frame cannot sequence them and the interaction is decoherent in that frame (the physics companion's reading of the high-energy regime). One does not observe the grammar; one pays to violate it and observes the remains.

Two consequences follow that are worth having. The first closes a circuit opened twice already in this paper. Establishing that structure is inert requires perturbing it, and the perturbation is the removal one wanted licensed (Remark (long form)); the dependents of a thing are held in other frames and cannot be surveyed from it (Remark 4.3); and now, the rule governing a frame is legible only where it is broken. These are one statement in three places: *the test and the change are the same operation, so knowledge of a boundary is bought by crossing it and there is no cheaper instrument.* Which also prices the instrument, since by the accelerator's own economics each further increment of boundary costs more than the last.

The second is a cost of handlers that is easy to miss because it looks like a benefit. *Every handler installed reduces what failure can teach.* A frame that coheres its failures to tidy values reports success and discloses nothing, so robustness and legibility are traded against one another rather than being two names for quality. That the same system is customarily made loud in development and quiet in production is this trade made deliberately — two frames wanting opposite policies over identical structure, for the reason given here rather than by convention.

5 Time

Time is not a substrate dimension. It is inferred by a frame from its own closure cycles, which is why a frame that runs no rules has none.

Proposition 5.1 (Time is inferred, not observed)**.** *No collapse presents a temporal coordinate. Succession is a relation* between *realised states, and a relation between states is not a datum*

of any one of them. A frame therefore acquires a time dimension only by inferring *it from the relations among what it holds. Consequently*

$$\boxed{\textit{a frame observes time} \implies \textit{that frame is capable of inference.}}$$

Proposition 5.2 (The unit of inferred time is a cycle, and requires closure)**.** *A frame does not count its individual changes — those are below its aperture — but its own* cycles*, each one traversal of a closed rule-loop. The unit of inferred time is therefore a cycle, not a change, and a cycle has no fixed length: each does a variable amount of change, so inferred time is non-uniform in underlying change (a nonlinear grounding, which Discrete Sufficiency already permits) and a fuller cycle reads as a longer moment (subjective dilation). A cycle exists only for a graph with* closure *— a graph without it runs to fixed point once and stops, one pass and no repeated tick — so having time requires closure as well as inference (Proposition 5.1): a one-shot computation, lacking a loop to iterate, has no time, and a frame's experienced "now" is one cycle of its own loop.*

Definition 5.3 (The present, structurally)**.** *Within a frame, the present is the collapse boundary at which every instantiated* undetermined *dimension relevant to that collapse has been determined:*

$$\boxed{U_{\mathcal{F}} = 0.}$$

$U_{\mathcal{F}}$ counts undetermined dimensions only. NULL dimensions are excluded — they are not variables. ***Undisclosed dimensions are also excluded****: they are already determined, and a frame's failure to hold them is an epistemic deficit rather than an unresolved collapse. A frame with outstanding undisclosed structure is still at its own present.*

6 Primitives, and the single prohibition

Definition 6.1 (Primitives)**.** *A graph's* primitives *are the operations it invokes without performing: the operator set its rules bottom out in. Every level has them, and having them is what being a level consists of — a graph with no primitives would have to perform everything it invokes, which is the regress this substrate does not contain. That is the whole of the definition, and in particular it says nothing about what a graph can learn of where they come from.*

Two different things can be meant by provenance being hidden, and running them together inverts the relation between them.

Deliberate exposure *— state maintained so that what runs on it can read it, as reflection does — is structure doing work for something other than the frame holding it, and is therefore excluded by well-formedness (Definition 3.2) in the ordinary way: it costs, and the frame supplying it gets nothing.*

Leakage *is the opposite case and is not excluded at all. A cache earns its retention many times over and leaks precisely* because *it does work; an arithmetic fast enough to overflow at a word boundary discloses the boundary by being fast; a page-sized performance cliff exists because the page size is doing something. Every leak in Proposition 3.4 is a consequence of efficient design rather than of poor design, so well-formedness does not prevent leakage — it* produces *it, and a structure that leaked nothing would have given up the optimisations that distinguish it from a uniform one, which is what constant-time implementations do and what they pay for.*

Remark 6.2 (What is required and what is optimisation). Only that rules run is compulsory for anything to *occur* — structure alone is what is required for there to be anything at all (Remark 1.6). Applying a delta is itself a rule, not a primitive operation beneath the rules, so the specific dynamics — inertia, the merge behaviour, closure — are contingent rule content, and a different r or Γ is a different physics rather than a violated law (this is why the initial and boundary conditions of a grounding are supplied separately from its laws; Section (long form)).

The two gates of Proposition 2.3, and even the retained standing structure itself, are *optimisations* on bare rule-running: the aperture grammar Γ_a bounds the input, the change grammar Γ_c sorts the realised output into what coheres and what is residual, and the retained structure buys free continuation. A substrate whose grammar admits everything still runs — effectively ungated, without momentum, wastefully; it is not a substrate *lacking* Γ, since Definition 1.3 defines the possibility structure through the grammar and a collapse needs something to select from — so they are what an efficient graph accretes under a budget, not constituents of graphhood. The retained structure specifically is what distinguishes a self-sustaining loop from a purely reactive one, since coasting is the cached structure.

Proposition 6.3 (Primitives are supplied, not intrinsic)**.** *A graph is data — nodes, relations, and rules — and the operations its rules bottom out in are supplied to it rather than contained in it. The same graph may therefore be run against different supplies of the same operator set, and nothing supplying an operator need be the substrate the graph describes or is "in." Where the work is done is a fact about that supply and not about the graph.*

Proposition 6.4 (A well-formed runner does not expose itself, and no level is self-transparent)**.** *The asynchronous, locally synchronous structure a frame observes (Remark (long form)) is the graph's* internal *view and does not fix its runner's actual mode. A hidden central runner may execute a graph serially, one change at a time, while the graph — lacking any access to the runner's clock — appears asynchronous and concurrent to itself. More generally, where a frame has no boundary to the substrate that runs it, its runner lies beyond the frame's epistemic boundary in exactly the sense of Section (long form).*

Two claims have to be separated here, and an earlier statement of this proposition ran them together by calling the runner unseeable rather than unseen. That a runner is hidden is contingent*. Nothing forbids a runner from exposing its own state to what it runs, and computational reflection does exactly that, as this paper acknowledges when placing itself against that literature (Section (long form)); a tower in which each level is available to the one above is a built artefact, not a thought experiment. A runner that does not expose itself is the* well-formed *case, and for the reasons well-formedness always turns on: exposed runner state is structure maintained for work the frame does not do, and each exposure needs a runner of its own to run the reading of it.*

What is not contingent is weaker and survives reflection: no level is self-transparent*. Reading is an operation whose product lands one level up (Remark 3.3), so a level cannot hold the reading of itself while being the level that is running. Reflection makes the levels explicit and pays for each; it does not terminate in a level transparent to itself. A frame may therefore be given sight of what runs it, and cannot be given sight of its own running. Since the rules are graph and a frame can read and rewrite them (Definition 2.1), while its own running is not graph it holds, the asymmetry is exact: a frame is transparent to itself in precisely one respect and opaque in precisely the other.* It can know what it is and cannot observe its being it. *Self-modification is available; self-observation at the level of the modifying is not.*

Proposition 6.5 (A primitive is another graph's running, exposed)**.** *Nothing supplying a primitive is a substance apart from graph data. An operator is the observable running of some other graph, which has primitives of its own, supplied in the same way. A graph's "primitives" are therefore the observable running of other graphs, exposed as an operator set, with no primitive runner at the base. A single chain — code over machine code over microcode over silicon over electromagnetism over quantum physics — is one path through that provision, and a common one, but not its shape: a process takes arithmetic from its processor, file operations from a kernel that relays them to a device, and remote calls from a machine that is not beneath it in any sense. Fan-out and relay are ordinary, and the tower is the degenerate case in which provision happens to be a chain. The regress bottoms not at a floor but at the epistemic frontier of Section (long form): below the lowest measured level is the unclaimed pre-relational substrate.*

The account forbids one thing: a frame cannot get outside itself. It cannot be transparent to itself at the level it is running at, cannot measure its own throughput, and cannot hold a view from nowhere. That prohibition has a formal analogue — a language of sufficient strength cannot define its own truth predicate, and must be given it from a metalanguage — and the analogue is proved, though the mapping from frames to formal systems is argued rather than proved. What is forbidden is complete self-capture and not partial self-modelling, which is why reflective towers work and a frame can know a great deal about itself.

7 Carvings

Since the rules draw the boundary and the rules are contingent, no carving is privileged. That does not make the notion vacuous: an ill-formed cut costs more writes, so a frame under a budget tends to be running cheaper cuts, and the cuts that look like joints are the cheap ones rather than the real ones.

Proposition 7.1 (Convergence is observed, not enforced)**.** *Two frames doing the same work converge on similar cuts as each becomes better formed, because the cheap cut for a given task is nearly unique. This is a tendency of cost and not the discovery of a privileged boundary, and the two are distinguishable in principle: a privileged boundary would* enforce *convergence, and enforcement would extend to the hard cases; cost-shaped cuts converge where the work is shared and come apart where it is not. The step from shared work to similar cuts assumes the cheap cut for a task is nearly unique, which is an empirical posit and not a result of this account; it is labelled again where it is used (Proposition 7.2).*

Proposition 7.2 (Convergence under compression)**.** *A structure operating under a constraint, and under pressure to spend nothing on what does no work, approaches the arrangements that constraint forces — asymptotically, and only over the region the constraint touches. Two such structures therefore come to resemble each other without either having access to the other, because both are downstream of the same forcing rather than converging on a target.*

Hypothesis 7.3 (Located degradation)**.** *Where a classification's purpose can be specified independently and in advance of inspecting its failures, the failures concentrate near the boundary of that purpose more than a uniform distribution over the domain would produce. The claim is comparative and is not that failures occur nowhere else.*

8 Falsifiability

The substrate risks little by design and the groundings carry the risk. Below are the commitments attached to the material this abridgement states. They are a subset: the long paper carries the full set, including those for results not stated here, and the generator reports the difference on every build so the omission stays visible rather than becoming a claim.

1. [SHAPE] **Minimal graph sufficiency.** Fails if basic realised graph dynamics require a primitive beyond (g, r, Γ) rather than represented or relational structure.

2. [SHAPE] **Discrete sufficiency.** Fails if a claimed observable consequence cannot be preserved by any discrete representation, including dynamically varying quantisation, but is preserved by a genuinely non-discrete primitive.

3. [SHAPE] **Antecedent-state principle.** Fails if a realised transition can require as antecedent a fact that exists only as consequence of that same transition, absent prior fixed-point structure.

4. [SHAPE] **Graph-law determinism.** Fails if two genuinely identical, causally complete antecedent configurations produce different realised successors without any additional causally operative distinction. Variation observed only from a frame that omits determining dimensions does not constitute such a falsification.

5. [DETAIL] **Located degradation.** Fails if, for classifications whose purpose is specifiable in advance of inspecting their failures, failure cases are distributed no differently near the purpose boundary than across the rest of the domain (Hypothesis 7.3).

6. [SHAPE] **Convergence is not enforced.** Fails if convergence between independently developed classifications of one domain extends to the cases where the classifications' purposes differ, which is what a privileged boundary would produce and cost-shaped convergence would not (Proposition 7.1). Marked [SHAPE] on reconsideration: a privileged boundary is not a mechanism this account could replace, since the contingency of the rules that draw a boundary is what Section (long form) rests on and what Axiom (long form) asserts one level up. Exhibiting one refutes rather than revises.

7. [SHAPE] **Provenance is never closed by inference.** Marked [SHAPE] on reconsideration: this is the single prohibition of Section (long form) in one of its operational forms, so a frame that closed the question would have got outside itself, which the account forbids rather than merely models. Not that primitives fail to leak — they do leak, and Proposition 3.4 says so. This fails if a frame can *settle* the provenance of its own operator set from inside: if some in-frame procedure establishes, rather than merely evidences, whether its operators are self-supplied, how many distinct structures supply them, or whether a serial hidden supply underlies an apparently asynchronous frame The escape is not that further primitives were used, which is unavoidable, but that their provenance is open in the same way, so no procedure closes the question rather than relocating it.

8. [DETAIL] **Solutions are nearly unique under a binding constraint.** The posit under Propositions 7.1 and 7.2. Fails if structures optimising hard against the same binding constraint, over the same region, are exhibited arriving at arrangements that do not resemble each other — in which case constraint narrows the space without selecting within it, and both propositions weaken to that.

9. [SHAPE] **Decoherence is one operation.** Fails if the manner in which an inadmissible write fails is a property of what was attempted rather than of the frame's rules — that is, if one frame under one rule set yields structurally distinct failure modes for two inadmissible writes, with the difference not attributable to that frame's own policy (Remark 4.1). Different outcomes across *different* frames do not bear on this; that is what the claim predicts.

10. [DETAIL] **A swallowed failure is relocated, never closed.** Not that a frame cannot detect its own handler discarding a failure — it can, by holding a record of the attempt independent of the handler's report, and that record is ordinary paid structure. This fails if some in-frame audit *settles* the question rather than moving it: if a frame can establish that no handler of its own has misreported, without that audit being itself a policy subject to the same abuse one level in (Remark 4.2).

11. [SHAPE] **Dependents are not held by the referent.** Fails if realised graph dynamics require a structure to carry the set of things depending on it — back-references primitive rather than installed — so that the consequences of removing a structure are recoverable from that structure alone, without querying the frames holding the references and without paid machinery (Remark 4.3). A substrate in which deletion is always clean falsifies this, and would also remove the distinction between an absence and a vacancy.

12. [SHAPE] **Termination follows supply, not containment.** Fails if containment alone determines it — if a subgraph must stop when the graph containing it stops, even where its primitives are supplied from beyond that graph (Remark 4.4). The partial case carries its own clause: fails if a frame losing part of its supply must either stop or continue correctly, rather than running with an incomplete operation set and undefined behaviour, or if such a frame can enumerate what it has lost from inside — which would also falsify the provenance commitment below.

13. [SHAPE] **A boundary is known only by crossing it.** Fails if a frame can *settle* the location of its own admissibility boundary without any write that crosses it. Extrapolation from inside the admissible region evidences a boundary and is not at issue; establishing one would be (Remark 4.5). This is the epistemic member of the same family as the provenance commitment below, and fails in the same manner — by a procedure that closes the question rather than relocating it.

14. [DETAIL] **Robustness trades against legibility.** Fails if a policy can bound a frame's residuals without reducing what its failures disclose about its grammar — a handler that both absorbs and reports with no loss (Remark 4.5). Cheaply testable in software, and marked [DETAIL] because the account survives its loss with the trade weakened to a tendency.